
Livrable

Python Project

Auteurs:

Jahdiel KINVI
Mourad SAID SOILIH

Enseignant:

Matthieu LARBOULLET

Sommaire

1. Introduction.....	4
1.1. Contexte.....	4
1.2. Problématique.....	5
2. Analyse du site et des données.....	5
2.1. Description du site.....	5
2.1.1. Sous sous partie 1.....	6
3. Code principal.....	6
4. Problématiques rencontrées.....	8
5. Analyse des données & Visualisations.....	8
6. Conclusion.....	8
6.1. Réponse à la problématique.....	8
6.2. Limites et améliorations possibles.....	9

Liste des sigles

- CUR : Équipe actuelle du joueur (l'équipe dans laquelle il joue en ce moment).
- PpG (Points per Game) : Moyenne de points marqués par match.
- RpG (Rebounds per Game) : Moyenne de rebonds pris par match.
- ApG (Assists per Game) : Moyenne de passes décisives par match.
- SpG (Steals per Game) : Moyenne d'interceptions par match.
- BpG (Blocks per Game) : Moyenne de contres par match.
- TOpG (Turnovers per Game) : Moyenne de ballons perdus par match.
- USG% (Usage Rate) : Pourcentage des actions offensives de l'équipe utilisées par le joueur quand il est sur le terrain. Plus le % est élevé, plus le joueur a la balle et prend des tirs.
- eFG% (Effective Field Goal Percentage) : Pourcentage de réussite ajusté : les tirs à 3 points comptent plus que les tirs à 2 points. Permet de mieux mesurer l'efficacité au tir.

1. Introduction

1.1. Contexte

Présentation rapide du site choisi

Sites finalement utilisés :

- HoopsHype : <https://hoopshype.com/salaries/players/2024-2025/>
- NBA Stuffer : <https://www.nbastuffer.com/2024-2025-nba-player-stats/>

Site initialement initialement prévu (abandonné) :

- Basketball Reference : <https://www.basketball-reference.com/>

Pourquoi ces sites ?

Basketball Reference était la première source principale envisagée (pour extraire les statistiques) en plus de Hoopshype (pour extraire les salaires), car elle est la référence pour les statistiques NBA complètes et historiques. Le site présentait un HTML propre, des tables faciles à scraper et des stats avancées complètes. L'architecture et les données sont propres. Cependant, après plusieurs tentatives différentes de scraping, j'ai rencontré des blocages techniques majeurs (voir section 4- Problématiques) qui m'ont contraint à pivoter vers deux sources alternatives :

1. Hoopshype : Base de données spécialisée dans les salaires NBA, régulièrement mise à jour, avec des données contractuelles détaillées (options d'équipe, options joueur, etc.). Site présentant un html simple et accessible et des classements clairs.
2. NBA Stuffer : Plateforme proposant des statistiques avancées (points, assists, rebonds, efficacité, usage rate, etc.). Ce site complète parfaitement Hoopshype en fournissant des métriques de performance nécessaires à l'analyse

Le croisement de ces deux sources permet d'analyser la relation entre rémunération et performance sportive, un sujet central dans l'économie du sport professionnel.

Type de données disponibles

1. Hoopshype :
 - Salaires des joueurs (saison 2024-25)
 - Identifiants joueurs et équipes
 - Options contractuelles (Team Option, Player Option, Two-Way)
 - Rang salarial
2. NBA Stuffer :
 - Statistiques offensives (points, assists, rebonds par match)
 - Statistiques d'efficacité (eFG%, 3P%, USG%, ORtg, DRtg)
 - Statistiques défensives (steals, blocks)
 - Métriques avancées (PER, Win Shares estimées)

1.2. Problématique

Question claire et mesurable

Les joueurs NBA les mieux payés sont-ils aussi les plus performants durant la saison régulière 2024-2025 ?

L'objectif est donc de vérifier si l'argent suit la performance, ou si certains contrats sont clairement déconnectés de la réalité sportive.

Formulation quantitative

On mesurera cette problématique via :

- le coefficient de corrélation (r) entre salaire et points par match :
 - si $r = 0 \Rightarrow$ aucune corrélation
 - si $r \neq 0 \Rightarrow$ corrélation significative :
 - dans ce cas on supposera que le seuil de significativité $p < 0.05$
- le ratio Performance / Salaire : points par M\$ (Million de dollars)
 - Identification des meilleurs deals (Best Deals : ratio > 30)
 - Identification des joueurs surpayés (ratio < 0.3)
- l'indicateur SES (Salary Efficiency Score) :
 - Composite score = Points + Assists + Rebonds
 - SES élevé $\Rightarrow$ Performance élevée à faible coût

Hypothèses éventuelles

- Hypothèse 1 : Les meilleurs joueurs offensifs sont aussi parmi les mieux payés (corrélation positive forte)
- Hypothèse 2 : Les joueurs en contrat rookie (< 5 M\$) ont un rapport performance/salaire largement supérieur.
- Hypothèse 3 : Une part non négligeable des joueurs (avoisinant les 10%) est soit surpayée, soit sous-payée, par rapport à leur production réelle.

2. Analyse du site et des données

2.1. Description du site

Structure générale

- Page HTML statique, rendu côté serveur. Extraction simple avec requests et BeautifulSoup
- Tableau unique contenant tous les joueurs (626 lignes)
- URL : ``https://hoopshype.com/salaries/players/2024-2025/``
- Classe CSS principale : `` .hh-salaries-ranking-table ``

Pagination :

- Pagination numérotée classique

- Affichage : environ 25-50 joueurs par pages

2.1.1. Sous sous partie 1

Liste des variables collectées

Type de chaque variable

Exemple :

Mourad (HoopsHype)		
Variable	Description	Type
Rank	Rang salarial	int
Player_ID	Identifiant joueur	int
Player	Nom du joueur	string
Team_ID	Identifiant équipe	int
Team	Nom de l'équipe	string
Salary	Salaire brut 2024-25	int
Salary_Millions	Salaire en millions	float
Team_Option	Option d'équipe	bool
Player_Option	Option joueur	bool
Two_Way	Contrat Two-Way	bool

Jahdiel (NBA Stuffer)		
Variable	Description	Type
player	Nom du joueur	string
pts	Points par match	float
ast	Passes décisives par match	float
trb	Rebonds par match	float
per	Player Efficiency Rating	float
ws	Win Shares	float
bpm	Box Plus Minus	float
vorp	Value Over Replacement Player	float
clean_name	Nom normalisé pour fusion	string

3. Code principal

Explication de la méthodologie principale.

- Script de récupération HTML ou JSON
 - Script de récupération HTML (HoopsHype)

```
import requests

from bs4 import BeautifulSoup

import pandas as pd

def scrape_hoopshype():

    url = "https://hoopshype.com/salaries/players/2024-2025/"

    headers = {'User-Agent': 'Mozilla/5.0'}

    response = requests.get(url, headers=headers)

    soup = BeautifulSoup(response.text, 'html.parser')

    players = []

    table = soup.select_one('table.hh-salaries-ranking-table')

    for row in table.select('tbody tr'):

        cols = row.find_all('td')

        players.append({

            'Player': cols[1].text.strip(),

            'Salary': int(cols[2].text.replace('$', '').replace(',',''),

            'Team_Option': 'Team' in cols[3].text,

            'Player_Option': 'Player' in cols[3].text
```

```

    })

df = pd.DataFrame(players)

df['Salary_Millions'] = df['Salary'] / 1_000_000

df.to_csv('hoopshype_salaries_2025.csv', index=False)

return df
'''

```

Il se connecte au site avec requests -> Lit la page HTML avec BeautifulSoup -> Récupère le tableau des salaires -> il convertit le tout en tableau (pandas) -> il ajoute le salaire en millions -> et enregistre le fichier hoopshype_salaries_2025.csv

- Fonction d'extraction

```

response = requests.get(url, headers=headers)

soup = BeautifulSoup(response.text, 'html.parser')

players = []

table = soup.select_one('table.hh-salaries-ranking-table')

```

- Extraction de données

```

players.append({

    'Player': cols[1].text.strip(),

    'Salary': int(cols[2].text.replace('$', '').replace(',', '')),

    'Team_Option': 'Team' in cols[3].text,

    'Player_Option': 'Player' in cols[3].text

})

```

- Nettoyage ou conversion des types


```
'Salary': int(cols[2].text.replace('$', '').replace(',', '')),
```

4. Problématiques rencontrées

- Problème 1 : Blocage Basketball Reference (source initiale)
Basketball étant ma première source envisagée car elle combine salaires ET statistiques sur une seule page. Après implémentation du scraper, j'ai rencontré le blocage systématique : Erreur 403 Forbidden après 2-3 requêtes.
- Solution envisagée (échec) : J'ai tenté d'utiliser les Headers User-Agent en vain. Le site utilise **CloudFlare** qui représente une protection anti-scraping, et détecte donc les Headers User-Agent. Je l'ai trouvé en récupérant le html :

```
<html>
<head>
<title>Just a moment...</title>
<!-- CloudFlare Challenge -->
</head>
<body>
<noscript>Enable JavaScript</noscript>
<!-- Captcha iFrame -->
</body>
</html>
```

- Solution finale : Abandon de Basketball Reference et pivot vers NBA Stuffer
 - Avantage imprévu : Sources plus récentes et mieux maintenues pour la saison 2024-25

- Problème 2 : Doublons dans Hoopshype
Certains joueurs apparaissaient plusieurs fois dans le dataset HoopsHype (626 lignes initiales -> 508 joueurs uniques après nettoyage)

Exemple concret :

Anthony Davis	\$54,126,450	False	False	False
Anthony Davis	\$54,126,450	True	False	False ← Team Option

Hoopshype liste parfois plusieurs lignes contractuelles pour un même joueur (parce que le même peut être transféré dans 2 franchises au cours d'une seule et même saison régulière, ou peut être coupé au cours de la même saison) :

Ligne 1 : Salaire garanti

Ligne 2 : Salaire avec option d'équipe

Ligne 3 : Salaire partiel si coupé

- Solution finale : Procéder à un tri par salaire décroissant + suppression doublons :

```
df = df.sort_values('Salary_Millions', ascending=False)
```

```
df = df.drop_duplicates(subset='clean_name', keep='first')
```

On garde le contrat principal (salaire max) ; on simplifie l'analyse de sorte à avoir 1 joueur = 1 ligne

- Résultat : 82 doublons supprimés (626 joueurs à 544 joueurs)

- Problème 3 : Contenu dynamique NBA Stuffer

J'ai tenté de scraper NBA Stuffer avec requests :

```
response = requests.get("https://www.nbastuffer.com/...")
soup = BeautifulSoup(response.text, 'html.parser')
table = soup.select_one('table#player-stats')

print(table)
```

Le tableau était invisible dans le HTML initial. La page était vide.

Le tableau est généré par JavaScript après le chargement de la page HTML initiale.

requests récupère uniquement le HTML brut, sans exécuter le JavaScript.

De plus en voulant extraire le JSON :

```
import requests
```

```
response = requests.get("https://www.nbastuffer.com/api/stats/...")
```

→ 404 Not Found (l'API n'est pas publique)

Le site n'expose pas d'API publique. Les données sont peut-être chargées via des requêtes internes que seul leur JavaScript peut faire.

- La solution qui a été mise en place a été l'utilisation de Selenium WebDriver pour atteindre le rendu complet. Je l'ai d'abord téléchargé : `python -m pip install selenium webdriver-manager beautifulsoup4 pandas`

Grâce à ce site :

<https://www.pythoniaformation.com/blog/tutoriels-python-par-categories/automatiser-avec-python/comment-faire-web-scraping-selenium-python> j'ai immédiatement appris

à me servir de Selenium pour scraper.

- Résultat : J'ai extrait 653 joueurs avec succès

Exemples :

- Difficulté à cibler les bonnes balises HTML
- Pagination complexe
- Données mal formatées
- Problème d'encodage
- Erreurs 403 / blocage
- Temps de chargement

Vous devez expliquer :

- Le problème
- La cause
- La solution

5. Analyse des données & Visualisations

6. Conclusion

6.1. Réponse à la problématique

Problématique initiale :

Les joueurs NBA les mieux payés sont-ils aussi les plus performants durant la saison régulière 2024-25 ?

OUI, il existe une corrélation forte et statistiquement significative.

Techniquement ?

- Le coefficient $r = 0.802$ ($p < 0.001$)
 - corrélation forte ($0.7 < r < 0.9$)
 - hautement significative ($p < 0.05$)
- Une régression linéaire traduit par :
 - $R^2 = 0.643$ (64.3% de variance expliquée)
 - Equation : $\text{salaire} = 0.34 * \text{points} + 5.67$
 - Chaque point = +340 \$ de salaire
- Comparaison par tranche :
 - Joueurs > 40 M\$: moyenne 24.9 pts / match
 - Joueurs 15 - 40 M\$: moyenne 17.3 pts / match
 - Joueurs < 15 M\$: moyenne 8.1 pts / match
- Hypothèses validées :
 - h1 : Corrélation positive forte confirmée ($r = 0.802 > 0.7$)
 - h2 : Rookies présentant meilleurs ratios (moyenne 38.5 pts / M\$ vs 0.9 pts / M\$ global)
 - h3 : 12% des joueurs (62/508) sont sur/sous-payés (écart > 50%)

D'un point de vue interprétation économique ?

Les équipes NBA payent globalement leurs joueurs en fonction de leur production offensive. Cette relation forte suggère donc :

- une efficacité du marché NBA : salaires reflètent la valeur ajoutée (points)
- asymétrie d'information limitée : scouts évaluent correctement les performances
- des exceptions notables : les blessures, les contrats legacy, les rookies sous-payés

Conclusion finale

Les joueurs les mieux payés sont effectivement les plus performants avec une corrélation de 0.802. Chaque point supplémentaire vaut environ 340k\$ de salaire. Toutefois, des inefficiences de marché existent (rookies performants sous-payés, vétérans blessés sur-payés), créant des opportunités d'optimisation salariale pour les équipes.

6.2. Limites et améliorations possibles

6.2.1. Limites identifiées

1. Échantillon temporel limité : analyse sur 1 seule saison (2024-25) ; ne capture pas les tendances pluriannuelles et une impossibilité d'analyser évolution salaire-performance
2. Variables manquantes : statistiques défensives absentes (steals, blocks, defensive rating) ; advanced metrics limitées (pas de PER, Win Shares, VORP, BPM) ; minutes jouées non collectées (biais joueurs blessés)
3. Biais de position : centers scorent naturellement moins mais ont valeur défensive élevée ; point guards ont plus d'assists (métrique non pondérée) ; pas de normalisation par position dans l'analyse
4. Contexte d'équipe ignoré : joueurs dans les équipes faibles peuvent avoir stats gonflées ; usage rate non pris en compte (joueurs uniques options offensives) ; qualité des coéquipiers impacte performances individuelles
5. Données d'équipes incomplètes : équipes "Unknown" dans dataset final ; impossibilité d'analyser masse salariale par franchise ; pas de visualisations par équipe
6. Limitation source Basketball Reference : abandon de la source la plus complète (protection anti-scraping) ; dépendance à 2 sources différentes (HoopsHype + NBA Stuffer) ; risque de désynchronisation entre sources

6.2.2. Améliorations possibles

1. Extension temporelle : scraper les 5 dernières saisons (2020-2025) ; analyser évolution salaire-performance dans le temps ; identifier joueurs en progression (contrats sous-évalués futurs)
2. Enrichissement variables : ajouter des stats défensives ([NBA.com](https://www.nba.com/api) API ou Basketball Reference contourné) ; collecter advanced metrics (PER, Win Shares, VORP, BPM) ; intégrer minutes jouées pour normaliser par temps de jeu
3. Modélisation prédictive : créer modèle random pour prédire les salaires ; identifier joueurs sous/sur-payés via les résidus du modèle ; prédiction salaires futurs basée sur trajectoire performance
4. Automatisation : un script planifié pour du scraping quotidien ; suivi en temps réel de l'évolution des stats durant la saison et l'actualisation des données sur les sites ; des alertes sur joueurs dépassant les performances prévues
5. Dashboard interactif : des interfaces pour des explorations interactives ; des filtres dynamiques (équipe, position, tranche salariale) ; comparaison des joueurs en temps réel.

NB : Le versioning des datasets et l'arborescence propre du projet se trouve ici :

https://github.com/ycoj2309/project-nba-scraping/blob/main/Rapport_final_projet_nba.md